

LAB# 11

PROJECTION AND DATA MODELING IN MONGODB

OBJECTIVE:

- To learn and implement projection in mongoDB.
- To learn and implement different types of data models in mongoDB.

THEORY:

PROJECTION:

MongoDB provides a special feature that is known as Projection. It allows you to select only the necessary data rather than selecting whole data from the document. For example, a document contains 5 fields, i.e.,

```
{  
name: "Roma",  
age: 30,  
branch: EEE,  
department: "HR",  
salary: 20000  
}
```

But we only want to display the name and the age of the employee rather than displaying whole details. Now, here we use projection to display the name and age of the employee. One can use projection with `db.collection.find()` method. In this method, the second parameter is the projection parameter, which is used to specify which fields are returned in the matching documents.

Syntax:

`db.collection.find({}, {field1: value2, field2: value2, ..})`

- If the value of the field is set to 1 or true, then it means the field will include in the return document.
- If the value of the field is set to 0 or false, then it means the field will not include in the return document.
- You are allowed to use projection operators, but `find()` method does not support following projection operators, i.e., `$`, `$elemMatch`, `$slice`, and `$meta`.
- There is no need to set `_id` field to 1 to return `_id` field, the `find()` method always return `_id` unless you set a `_id` field to 0

EXAMPLE:

We have a collection named 'authors' in the database 'books' and it looks like the following

```
> db.authors.find()
< {
  _id: ObjectId('658c3fc8a7b0e22012e6235a'),
  name: 'Author 1',
  nationality: 'Country 1',
  books: [
    'Book 1',
    'Book 2',
    'New Book'
  ]
}
{
  _id: ObjectId('658c3fc8a7b0e22012e6235c'),
  name: 'Author 3',
  nationality: 'Country 3',
  books: [
    'Book 5',
    'Book 6'
  ]
}
```

Now, if we want to view the names of the authors, the query will look like following:

```
> db.authors.find({}, {name:1})
< {
  _id: ObjectId('658c3fc8a7b0e22012e6235a'),
  name: 'Author 1'
}
{
  _id: ObjectId('658c3fc8a7b0e22012e6235c'),
  name: 'Author 3'
}
```

Here, we can notice that the id field is also displaying. To turn it off , the query will look like the following:

```
> db.authors.find({}, {name:1 , _id:0})
< {
  name: 'Author 1'
}
{
  name: 'Author 3'
}
```

RETURN ALL BUT THE EXCLUDED FIELDS

Instead of listing the fields to return in the matching document, you can use a projection to exclude specific fields. The following example which returns all fields except for the 'nationality' and the field in the matching documents:

```
> db.authors.find({}, {nationality:0})
< {
  _id: ObjectId('658c3fc8a7b0e22012e6235a'),
  name: 'Author 1',
  books: [
    'Book 1',
    'Book 2',
    'New Book'
  ]
}
{
  _id: ObjectId('658c3fc8a7b0e22012e6235c'),
  name: 'Author 3',
  books: [
    'Book 5',
    'Book 6'
  ]
}
```

DATA MODELING

Data modeling refers to the organization of data within a database and the links between related entities. Data in MongoDB has a **flexible schema model**, which means:

- [Documents](#) within a single [collection](#) are not required to have the same set of fields.
- A field's data type can differ between documents within a collection.

Generally, documents in a collection share a similar structure.

LINK RELATED DATA

When you design your data model in MongoDB, consider the structure of your documents and the ways your application uses data from related entities.

To link related data, you can either:

- Embed related data within a single document.
- Store related data in a separate collection and access it with a reference.

Embedded Data

Embedded documents store related data in a single document structure. A document can contain arrays and sub-documents with related data. These denormalized data models allow applications to retrieve related data in a single database operation.



Embedded data models let applications query related pieces of information in the same database record. As a result, applications require fewer queries and updates to complete common operations.

Use Cases

Use embedded data models in the following scenarios:

- You have "contains" relationships between entities. For example, a contacts document that contains an address. You have one-to-many relationships between entities. In these

relationships, the "many" or child documents are viewed in the context of the "one" or parent documents.

Embedding provides the following benefits:

- Better performance for read operations
- The ability to retrieve related data in a single database operation
- The ability to update related data in a single atomic write operation

EXAMPLE:

In this example, the "blog" collection has embedded documents within it. The "comments" field contains an array of comment documents, each with its own properties like "username," "text," and "timestamp."

```
> db.blog.insertOne({
  "title": "Sample Blog Post 1",
  "content": "This is the content of the blog post.",
  "author": "John Doe",
  "comments": [
    {
      "username": "Alice",
      "text": "Great post! Thanks for sharing.",
      "timestamp": ISODate("2024-01-08T12:00:00Z")
    },
    {
      "username": "Bob",
      "text": "I have a question. Can you elaborate on this point?",
      "timestamp": ISODate("2024-01-08T14:30:00Z")
    }
  ],
  "timestamp": ISODate("2024-01-08T10:00:00Z")
})
```

QUERYING EMBEDDED DOCUMENTS

In MongoDB, you can query embedded documents using dot notation to navigate through the fields. For example, to query for blog posts with a specific comment, you can use the dot notation to access the properties within the embedded array. Here are a few examples:

EXAMPLE#1

Find all blog posts with comments from a specific user:

```
> db.Blogs.find({"comments.username": "Alice"})
```

EXAMPLE#2

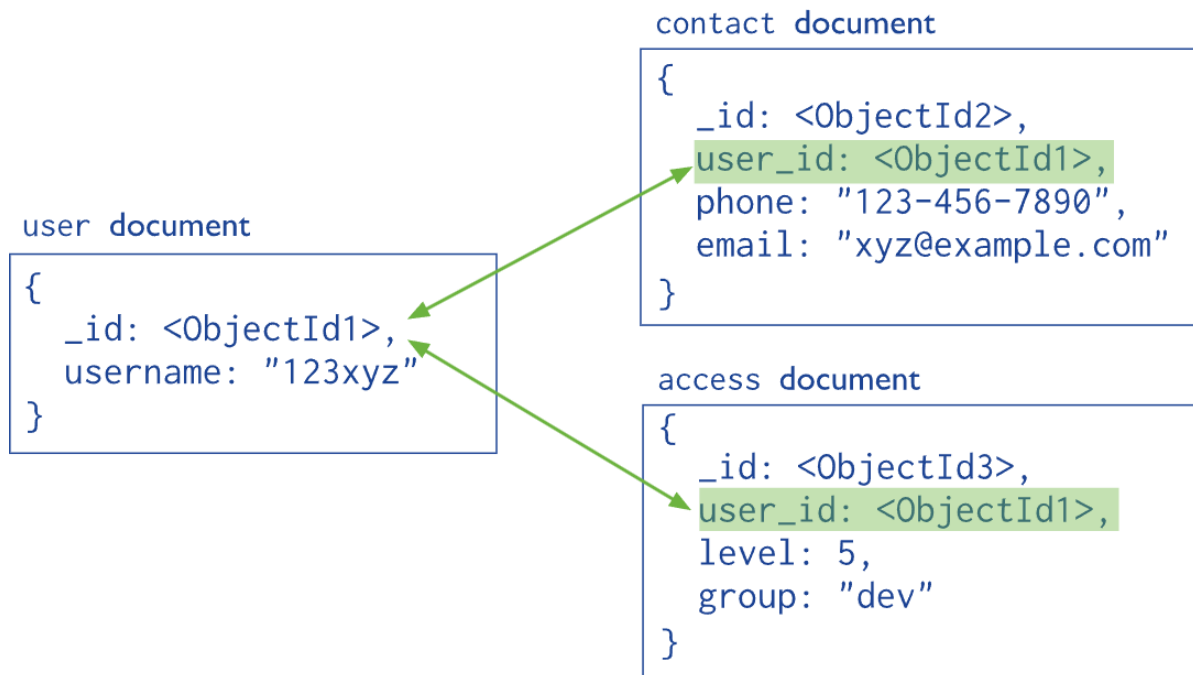
Find all blog posts with comments posted after a certain timestamp:

```
> db.Blogs.find({"comments.timestamp": {$gt: ISODate("2024-01-08T13:00:00Z")}})
```

REFERENCES

References store relationships between data by including links, called **references**, from one document to another. For example, a `customerId` field in an `orders` collection indicates a reference to a document in a `customers` collection.

Applications can resolve these references to access the related data.



USE CASES

Use references to link related data in the following scenarios:

Embedding would result in duplication of data but would not provide sufficient read performance advantages to outweigh the implications of the duplication. For example, when the embedded data frequently changes.

You need to represent complex many-to-many relationships or large hierarchical data sets.

The related entity is frequently queried on its own. For example, if you have employee and department data, you may consider embedding department information in the employee documents. However, if you often query for a list of departments, your application will perform best with a separate department collection that is linked to the employee collection with a reference.

EXAMPLE:

```
> db.createCollection("users")
< { ok: 1 }
> db.users.insertMany([
  {
    _id: 1,
    username: "john_doe",
    email: "john.doe@example.com",
    age: 25
  },
  {
    _id: 2,
    username: "jane_smith",
    email: "jane.smith@example.com",
    age: 30
  }
])
```

Create second collection named 'post' and insert documents

```

> db.createCollection("post")
< { ok: 1 }
> db.post.insertMany([
  {
    title: "Post 1",
    content: "This is the content of post 1.",
    user_id: 1
  },
  {
    title: "Post 2",
    content: "This is the content of post 2.",
    user_id: 2
  },
  {
    title: "Post 3",
    content: "This is the content of post 3.",
    user_id: 1
  }
])

```

QUERYING THE REFERENCED DOCUMENT

Let's follow the above example and retrieve the user-information corresponding to the posts. The query is a MongoDB aggregation query that demonstrates how to retrieve posts with associated user information by performing a left join between the "posts" and "users" collections. Let's break down the query step by step:

1)Lookup Stage

```

$lookup: {
  from: "users",
  localField: "user_id",
  foreignField: "_id",
  as: "user"
}

```

- The **\$lookup** stage is used for performing a left join with another collection.
- **from**: Specifies the target collection to join, in this case, "users."
- **localField**: The field from the input documents (in the "posts" collection) that will be used for matching, which is "user_id."
- **foreignField**: The field from the target documents (in the "users" collection) that will be used for matching, which is "_id."

- **as**: The name of the new array field that will contain the joined documents, which is "user."

2) Unwind stage

```
{  
  $unwind: "$user"  
},
```

The **\$unwind** stage is used to flatten the array created by the **\$lookup** stage. This is necessary because **\$lookup** creates an array even if there's only one matching document.

3) Project stage

```
{  
  $project: {  
    title: 1,  
    content: 1,  
    "user.username": 1,  
    "user.email": 1  
  }  
}
```

- The **\$project** stage is used to shape the output documents.
- It includes fields like "title" and "content" directly from the "posts" collection.
- It includes fields from the "user" array, such as "username" and "email," using dot notation.

The complete query and its output are as follows:

```
> db.post.aggregate([
  {
    $lookup: {
      from: "users",
      localField: "user_id",
      foreignField: "_id",
      as: "user"
    }
  },
  {
    $unwind: "$user"
  },
  {
    $project: {
      title: 1,
      content: 1,
      "user.username": 1,
      "user.email": 1
    }
  }
])
```

```
< {
  _id: ObjectId('659db0ae28e9fff72fe3acb9'),
  title: 'Post 1',
  content: 'This is the content of post 1.',
  user: {
    username: 'john_doe',
    email: 'john.doe@example.com'
  }
}
{
  _id: ObjectId('659db0ae28e9fff72fe3acba'),
  title: 'Post 2',
  content: 'This is the content of post 2.',
  user: {
    username: 'jane_smith',
    email: 'jane.smith@example.com'
  }
}
{
  _id: ObjectId('659db0ae28e9fff72fe3acbb'),
  title: 'Post 3',
  content: 'This is the content of post 3.',
  user: {
    username: 'john_doe',
    email: 'john.doe@example.com'
  }
}
}
```

LAB TASKS:

- 1) Study the following scenarios and suggest how the database should be modeled whether embedded or referenced? Support your answer with reasons.

Scenario 1: Blogging Platform

You are designing a MongoDB schema for a blogging platform. Each blog post can have multiple comments.

Scenario 2: E-commerce Website

You are building a database for an e-commerce website. Each product can have multiple reviews.

Scenario 3: Geographic Data

You are working on a system that stores geographic data, including countries, regions, and cities.

Scenario 6: Messaging App

You are developing a messaging application. Each conversation can have multiple messages.

- 2) Create collections for scenarios in task#1 and insert documents (as per your answer in task#1).
- 3) Apply retrieval operation in the collections created above, and show how the data is retrieved in referenced model as compared to embedded model (utilize projection).
- 4) Explore and implement *\$elematch* and *\$regex* methods and embed them in your program (in any of the above scenarios).
- 5) Answer the following questions:
 - What have you learned from the lab tasks?
 - What was the most challenging task and how did you overcome that challenge?